FernUniversität in Hagen
Faculty of Mathematics and Computer Science

AIG
Artificial Intelligence Group

An Investigation of a Notion of „Variable Forgetting“ that Minimizes Truth Values

Bachelor's Thesis

in partial fulfillment of the requirements for
the degree of Bachelor of Science (B.Sc.)
in Computer Science

submitted by
Christoph Kaplan

First examiner:	Prof. Dr. Matthias Thimm Artificial Intelligence Group
Advisor:	Kai Sauerwald Artificial Intelligence Group

Statement

Ich erkläre, dass ich die Bachelorarbeit selbstständig und ohne unzulässige Inanspruchnahme Dritter verfasst habe. Ich habe dabei nur die angegebenen Quellen und Hilfsmittel verwendet und die aus diesen wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht. Die Versicherung selbstständiger Arbeit gilt auch für enthaltene Zeichnungen, Skizzen oder graphische Darstellungen. Die Bachelorarbeit wurde bisher in gleicher oder ähnlicher Form weder derselben noch einer anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht. Mit der Abgabe der elektronischen Fassung der endgültigen Version der Bachelorarbeit nehme ich zur Kenntnis, dass diese mit Hilfe eines Plagiatserkennungsdienstes auf enthaltene Plagiate geprüft werden kann und ausschließlich für Prüfungszwecke gespeichert wird.

	Yes	No
I agree to have this thesis published in the library.	☐	☐
I agree to have this thesis published on the webpage of the artificial intelligence group.	☐	☐
The thesis text is available under a Creative Commons License (CC BY-SA 4.0).	☐	☐
The source code is available under a GNU General Public License (GPLv3).	☐	☐
The collected data is available under a Creative Commons License (CC BY-SA 4.0).	☐	☐

.....

(Place, Date) (Signature)

Abstract

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua.

Contents

1 Motivation

Propositional logic deals, among other things, with the satisfiability of sentences or the inference of sentences from a set of premises. A knowledge base may contain many different statements in various respects. To assert or entail sentences about specific phenomena or interest, it would be advantageous to work only with relevant knowledge to reduce computational complexity. In other words, non-relevant or irrelevant information in relation to the current interest could be "forgotten". Furthermore, knowledge bases change over time, and old knowledge might need to be contracted. To this end, an operation exists for eliminating variables within a proposition. As stated in [LR94] or [LLM03], we define variable forgetting inductively as follows:

Definition 1 (Skeptical Variable Forgetting) *Let F be a formula, p an atomic variable and S as Set of Variables. The* skeptical forgetting of p in F *$SkepForget(F, p)$, respectively, the* skeptical forgetting of S in F *$SkepForget(F, S)$, is given inductively by:*

- $SkepForget(F, \emptyset) = F$
- $SkepForget(F, \{p\}) = F[p/\top] \wedge F[p/\bot]$
- $SkepForget(F, p) = SkepForget(F, \{p\})$
- $SkepForget(F, S \cup \{p\}) = SkepForget(SkepForget(F, \{p\}), S)$

For some formulas F, this operation of variable elimination can lead to a notable reductive or drastic results. We start by considering an example where *Forget* behaves quite appropriate. Let a sentence F_1 be $a \wedge b$ and we choose to forget about b:

$$\begin{aligned} Forget(F_1, b) &= F_1[b/\top] \vee F_1[b/\bot] \\ &\equiv ((a \wedge \top) \vee (a \wedge \bot)) \\ &\equiv (a \vee \bot) \\ &\equiv a \end{aligned}$$

We can see, as shown above, that $Forget(F_1, b)$ is equivalent to a which seems to be the intuitive result when thinking of forgetting about b in F_1. Next, we consider forgetting of b in the formula $F_2 = a \vee b$

$$\begin{aligned} Forget(F_2, b) &= F_2[b/\top] \vee F_2[b/\bot] \\ &\equiv ((a \vee \top) \vee (a \vee \bot)) \\ &\equiv (\top \vee a) \\ &\equiv \top \end{aligned}$$

We observe that $Forget(F_2, b)$ is *tautological*, which can be seen as a rather drastic result when we choose to forget b only. A variation of a forget operation could be defined as follows:

Definition 2 (Skeptical Variable Forgetting) *Let F be a formula, p an atomic variable and S as Set of Variables. The* skeptical forgetting of p in F *$SkepForget(F, p)$, respectively, the* skeptical forgetting of S in F *$SkepForget(F, S)$, is given inductively by:*

- $SkepForget(F, \emptyset) = F$
- $SkepForget(F, \{p\}) = F[p/\top] \wedge F[p/\bot]$
- $SkepForget(F, p) = SkepForget(F, \{p\})$
- $SkepForget(F, S \cup \{p\}) = SkepForget(SkepForget(F, \{p\}), S)$

The idea is to use conjunction instead of disjunction. The consequences for our previous examples are the following:

$$\begin{aligned} SkepForget(F_1, b) &= F_1[b/\top] \vee F_1[b/\bot] \\ &\equiv ((a \wedge \top) \wedge (a \wedge \bot)) \\ &\equiv (a \wedge \bot) \\ &\equiv \bot \end{aligned}$$

$$\begin{aligned} SkepForget(F_2, b) &= F_2[b/\top] \vee F_2[b/\bot] \\ &\equiv ((a \vee \top) \wedge (a \vee \bot)) \\ &\equiv (\top \wedge a) \\ &\equiv a \end{aligned}$$

We observe that our alternative operation, at least in these examples, doesn't necessarily entail less drastic results. We obtain a contradiction in the first case and in the second case, our equivalent to a. It suggests a duality between the two operations.

2 SyntacticAnalysis

To explore the properties of our $SkepForget$ operation, we need to identify the space or dimensions in which the function operates. While introductory examples provided in the previous section shed light on the primary issue, it remains crucial to investigate the behavior of $SkepForget$ with more complex sentences. Do the outcomes predominantly correlate with the top layer of connectives in the syntax tree? What influence do deeper nested expressions exert on the final result?
To draw valuable conclusions it is necessary to select examples that can proof generality while beeing specific enough to not loose any valuable properties and characteristics.How to identify such examples ? What criteria are essential for their selection?
From the syntactical perspective we have sketched several dimensions for comparison: We need to examine the behavior of the two forget operations in relation — are they dual in nature or not? We need to consider complexity of sentences such as

negation and multiple binary connectives with deeper nestingComparing the forgetting of variables at different depths, for instance, for a sentence such as (a or b or c) we have three options.
Lets start our investigation with the following sentences:

$$\begin{array}{llll} F_{\alpha 1} & \equiv a & Forget(F_{\alpha 1}, a) & \equiv \top \quad SkepForget(F_{\alpha 1}, a) \equiv \bot \\ F_{\alpha 2} & \equiv \neg a & Forget(F_{\alpha 2}, a) & \equiv \top \quad SkepForget(F_{\alpha 2}, a) \equiv \bot \end{array}$$

In $F_{\alpha 1}$ and $F_{\alpha 2}$ we forget the whole sentence itself where $F_{\alpha 2}$ is negation of $F_{\alpha 1}$.In both cases the result is $\top$ for $Forget$ and $\bot$ for $SkepForget$ as if it would characterize the function fundamentally.Lets see what happens with more complex sentences.

$$\begin{array}{llll} F_{\beta 1} & \equiv (a \wedge b) & Forget(F_{\beta 1}, a) & \equiv b \quad SkepForget(F_{\beta 1}, a) \equiv \bot \\ F_{\beta 2} & \equiv \neg(a \wedge b) & Forget(F_{\beta 2}, a) & \equiv \top \quad SkepForget(F_{\beta 2}, a) \equiv \neg b \\ F_{\beta 3} & \equiv (a \vee b) & Forget(F_{\beta 3}, a) & \equiv \top \quad SkepForget(F_{\beta 3}, a) \equiv b \\ F_{\beta 4} & \equiv \neg(a \vee b) & Forget(F_{\beta 4}, a) & \equiv \neg b \quad SkepForget(F_{\beta 4}, a) \equiv \bot \end{array}$$

Here we can see that $Forget(F_{\beta 1}, a) \equiv \neg(SkepForget(F_{\beta 2}, a))$
With $F_{\beta 1} \equiv \neg F_{\beta 2}$ we get $Forget(F_{\beta 1}, a) \equiv \neg SkepForget(\neg F_{\beta 1}, a) \neg F_{\beta 1} \equiv F_{\beta 2}$ and $Forget(\neg F_{\beta 2}, a) \equiv \neg SkepForget(F_{\beta 2}, a)$
More generaly we can establish a relation that: $Forget(F, a) \equiv \neg SkepForget(\neg F, a)$ and $Forget(\neg F, a) \equiv \neg SkepForget(F, a)$
Lets focus more on the logical connectives:

$$\begin{array}{llll} F_{\gamma 1} & \equiv (a \wedge b) & Forget(F_{\gamma 1}, a) & \equiv b \quad SkepForget(F_{\gamma 1}, a) \equiv \bot \\ F_{\gamma 2} & \equiv (a \vee b) & Forget(F_{\gamma 2}, a) & \equiv \top \quad SkepForget(F_{\gamma 2}, a) \equiv b \end{array}$$

In $F_{\beta 1}$ and $F_{\beta 2}$ we forget only one variable of a binary connective. Since logical connectives are commutative, forgetting the other variable would not give us new information.We observe a dualistic behavior. Our forget operations deliver either truth values or the remaining variable b of the original sentence.A necessary condition seems to be the connective itself. We can observe that Forget with $\wedge$ gives us $\bot$ where in combination with $\vee$ we receive b. For SkepForget its the other way arround, also the truth value is $\top$ wich is the oppositeBottom line, with the 4 results, we loose exactly the variable, wich we expect to loose. And in the other two cases we loose more information thatn we expectLets see if this holds true if we look are more complex/longer sentences:

$$\begin{array}{llll} F_{\delta 1} & \equiv (a \wedge (c \wedge d)) & Forget(F_{\delta 1}, a) & \equiv (c \wedge d) \quad SkepForget(F_{\delta 1}, a) \equiv \bot \\ F_{\delta 2} & \equiv (a \wedge (c \vee d)) & Forget(F_{\delta 2}, a) & \equiv (c \vee d) \quad SkepForget(F_{\delta 2}, a) \equiv \bot \\ F_{\delta 3} & \equiv (a \vee (c \wedge d)) & Forget(F_{\delta 3}, a) & \equiv \top \quad SkepForget(F_{\delta 3}, a) \equiv (c \wedge d) \\ F_{\delta 4} & \equiv (a \vee (c \vee d)) & Forget(F_{\delta 4}, a) & \equiv \top \quad SkepForget(F_{\delta 4}, a) \equiv (c \vee d) \end{array}$$

If we add more complexity via one additional connective.It seems that, due to the inductive or recursive structure of propositional logic we still get the same dual-

istic result as in the previous example. As in the previous example, in half of the cases we loose exactly what we expect, and in the other half we loose more information.Just to be on the safe side, lets see if it still holds when we add one more variable

$F_{\theta 1}$	$\equiv ((a \wedge b) \wedge (c \wedge d))$	$Forget(F_{\theta 1}, a)$	$\equiv (b \wedge (c \wedge d))$	$SkepForget(F_{\theta 1}, a)$	$\equiv \bot$
$F_{\theta 2}$	$\equiv ((a \wedge b) \wedge (c \vee d))$	$Forget(F_{\theta 2}, a)$	$\equiv (b \wedge (c \vee d))$	$SkepForget(F_{\theta 2}, a)$	$\equiv \bot$
$F_{\theta 3}$	$\equiv ((a \wedge b) \vee (c \wedge d))$	$Forget(F_{\theta 3}, a)$	$\equiv (b \vee (c \wedge d))$	$SkepForget(F_{\theta 3}, a)$	$\equiv (c \wedge d)$
$F_{\theta 4}$	$\equiv ((a \wedge b) \vee (c \vee d))$	$Forget(F_{\theta 4}, a)$	$\equiv (b \vee (c \vee d))$	$SkepForget(F_{\theta 4}, a)$	$\equiv (c \vee d)$
$F_{\theta 5}$	$\equiv ((a \vee b) \wedge (c \wedge d))$	$Forget(F_{\theta 5}, a)$	$\equiv (c \wedge d)$	$SkepForget(F_{\theta 5}, a)$	$\equiv (b \wedge (c \wedge d))$
$F_{\theta 6}$	$\equiv ((a \vee b) \wedge (c \vee d))$	$Forget(F_{\theta 6}, a)$	$\equiv (c \vee d)$	$SkepForget(F_{\theta 6}, a)$	$\equiv (b \wedge (c \vee d))$
$F_{\theta 7}$	$\equiv ((a \vee b) \vee (c \wedge d))$	$Forget(F_{\theta 7}, a)$	$\equiv \top$	$SkepForget(F_{\theta 7}, a)$	$\equiv (b \vee (c \wedge d))$
$F_{\theta 8}$	$\equiv ((a \vee b) \vee (c \vee d))$	$Forget(F_{\theta 8}, a)$	$\equiv \top$	$SkepForget(F_{\theta 8}, a)$	$\equiv (b \vee (c \vee d))$

Surprisingly we dont get exactly the same behaviour as before. The dualistic overall picture remains.But for $F_{\theta 5}$ and $F_{\theta 6}$ on the Forget side and $F_{\theta 3}$ and $F_{\theta 4}$ on the SkepForget side the result is structurally different from what we expected compared to the previous example.Because, as in the previous example, in half of the cases we loose exactly what we want to loose but in the other half it is more differentiated. We loose some but we dont loos as much as in the previous. But we made a little mistake here:Imagine a from the previous example is now replaced with with $(a \circ b)$, we now forget only a and not $(a \circ b)$ wich we should in order to make the same move as beforebut we can see that within the half of the cases in wich we loose information again we can divide this in half and observe that we only lost half of that in drastic way, meaning reuslt is only truth valuesAs we can only forget atomic variables, we cant make this comparison as we would need to forget $(a \circ b)$.We now would need to look at forgetting different variables in same sentences:note: also a is now deeper nested vs before it was top layer

For a sentence $a \wedge (b \vee c)$ we have 3 options, but only 2 layers in depth, considering commutativity of logical connectives we can reduce this to 2 options.

$F_{\phi 1}$	$\equiv (a \wedge (b \wedge c))$	$Forget(F_{\phi 1}, a)$	$\equiv (b \wedge c)$	$SkepForget(F_{\phi 1}, a)$	$\equiv \bot$
$F_{\phi 2}$	$\equiv (a \wedge (b \vee c))$	$Forget(F_{\phi 2}, a)$	$\equiv (b \vee c)$	$SkepForget(F_{\phi 2}, a)$	$\equiv \bot$
$F_{\phi 3}$	$\equiv (a \vee (b \wedge c))$	$Forget(F_{\phi 3}, a)$	$\equiv \top$	$SkepForget(F_{\phi 3}, a)$	$\equiv (b \wedge c)$
$F_{\phi 4}$	$\equiv (a \vee (b \vee c))$	$Forget(F_{\phi 4}, a)$	$\equiv \top$	$SkepForget(F_{\phi 4}, a)$	$\equiv (b \vee c)$

$F_{\phi 1}$	$\equiv (a \wedge (b \wedge c))$	$Forget(F_{\phi 1}, c)$	$\equiv (a \wedge b)$	$SkepForget(F_{\phi 1}, c)$	$\equiv \bot$
$F_{\phi 2}$	$\equiv (a \wedge (b \vee c))$	$Forget(F_{\phi 2}, c)$	$\equiv a$	$SkepForget(F_{\phi 2}, c)$	$\equiv (a \wedge b)$
$F_{\phi 3}$	$\equiv (a \vee (b \wedge c))$	$Forget(F_{\phi 3}, c)$	$\equiv (a \vee b)$	$SkepForget(F_{\phi 3}, c)$	$\equiv a$
$F_{\phi 4}$	$\equiv (a \vee (b \vee c))$	$Forget(F_{\phi 4}, c)$	$\equiv \top$	$SkepForget(F_{\phi 4}, c)$	$\equiv (a \vee b)$

taking on the perspective from the variable to forget we can see that half of the cases we loose only what we expect.

a is in the top layer, here will get themost drastic results where the deeper nested variable c is were we get the less drastic results, we loose less information

It seems that we loose information.

2.1 Behavior of Truth Values in Abstract Syntax Trees

The behavior of truth values presents notably insights when considering their interactions with logical connectives. We know that $\top \wedge \bot \equiv \bot$ and $\top \vee \bot \equiv \top$.

To investigate the behaviour abstract syntax trees are a great way to visualize sentences.To get a good overview we will try any permutation of connectives over a certain variable amount n wich is $2^(n-1)$ possibilities.

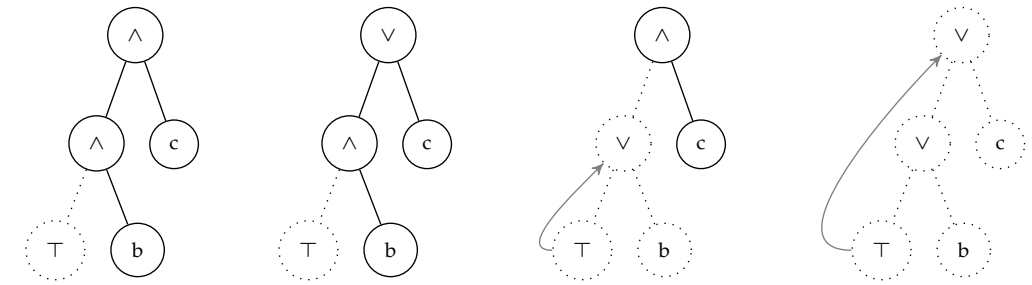

Figure 1: $\top$ propagation

In figure 3 we can observe the propagation of truth values, in this case $\top$. Starting from the leafes @$\top$ effectively "absorbs" all siblings in conjunction with $\vee$ and stops propagating at $\wedge$.

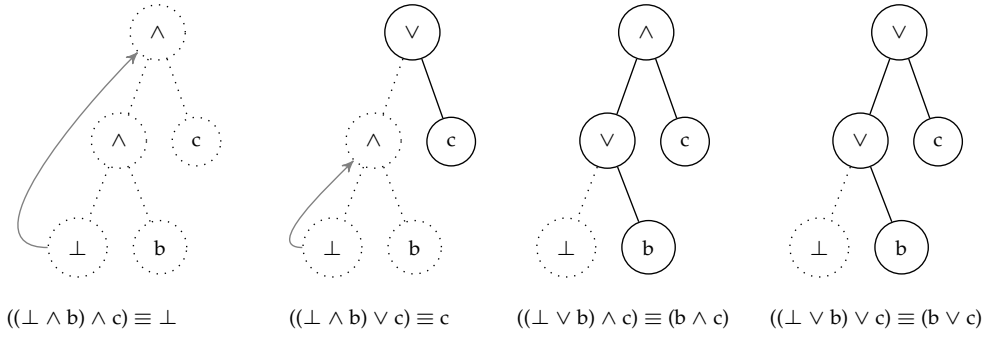

Figure 2: $\bot$ propagation

Figure 2 shows, dual to 3, how $\bot$ "annihilates" all siblings when paired with a logical $\wedge$ and halts at $\vee$. This dynamic interaction implies that truth values possess the capability to selectively "eliminate" variables and entire propositions.

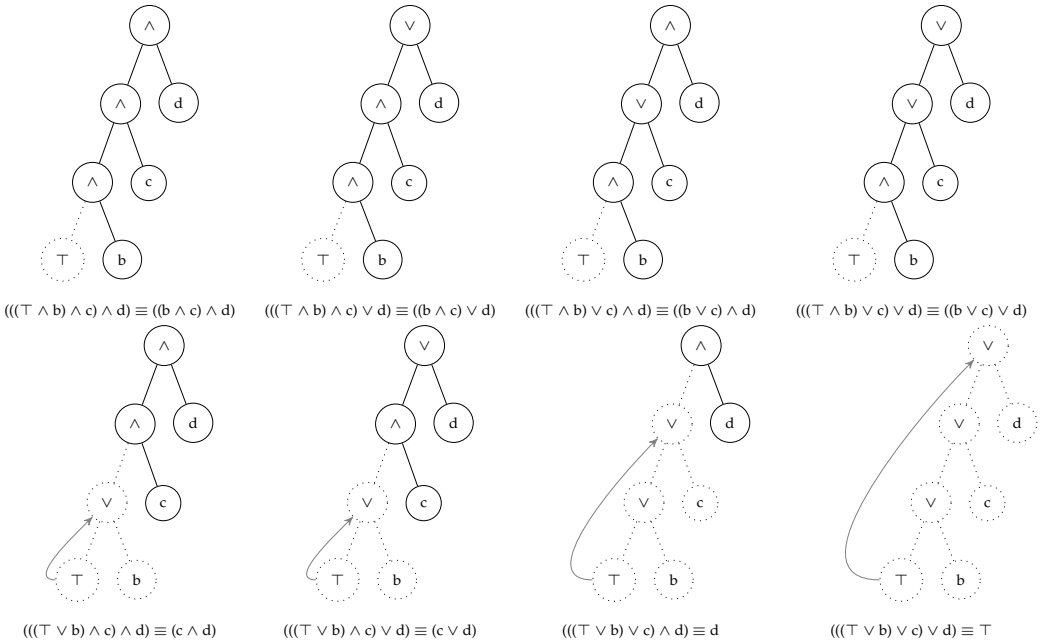

Figure 3: $\top$ propagation

Lets say V is the Set of variables in a Sentence, and $L \in \mathbb{N} \geq 0$ is the layer in the syntax tree. We can denote the possible amount of truth values for any sentence over all possible connectives as $TVA = 2^{|V|-1}/2^L$,
the partially lost information as $PL = (2^{|V|-1}/2) - TVA$
and the fully kept information as $K = (2^{|V|-1}/2)$

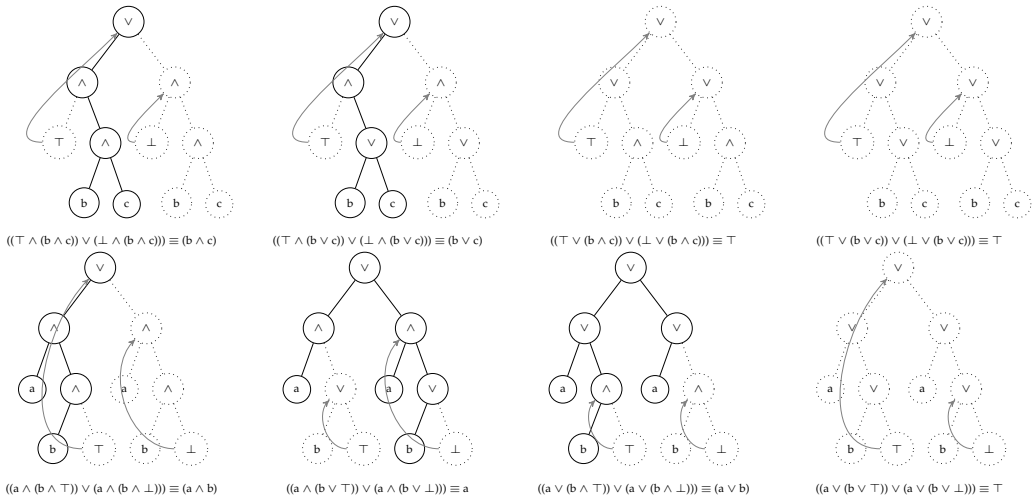

Figure 4: ddddd

3 SemanticInvestigation

In [LLM03], the authors denote two semantic functions, Force and Switch, as follows: Given in interpretation w and a literal l, we let Force(w,l) denote the interpretation that gives the same truth value as w for all variables except the variable of l, and such that Force(w,l) $\models$ l. Meaning Force(w,l) is the interpretation that satisfying l that is closest to w, If w $\models$ l then Force(w,l)=w.
$Switch(w, l)$ denotes the interpretation that maintains the same truth value as w for all variables except x, just like Force, but assigns to x the opposite truth value given by w.

3.1 Properties of Forget

In [LR94] the authors state the following properties of Forget:

- $T \models forget(T, p)$
- For any sentence ϕ, $T \models \phi \downarrow piff forget(T; p) \models \phi \downarrow p$ (works for FOL, not pl ???)

[EKI19]

- $M_1 \sim_p M_2, M_1 and M_2$ agree on all variables except possibly p

In [LLM03] the authors state the following properties of Forget:

- Proposition 14: The set of models of $ForgetLit($
  $Sigma, \{l\})$ can be expressed as:
  Corollary 5: $Mod(ForgetLit(\Sigma, \{l\})) = Mod(\Sigma) \cup \{Force(w, -l) | w \models \Sigma\} = \{w | Force(w, l) \models \Sigma\}$
- $Mod(Forget(F, x)) = Mod(F) \cup \{Switch(w, x) | w \models F\}$

$Int_{\Sigma_2}(Forget(F, a)) = Mod_{\Sigma_1}(a)$
$\forall w : w \models a, w(Forget(F, a)) = w(F)$
$\forall w : w \not\models a, w(Forget(F, a)) = Force(w, A)(F)$

3.2 Properties of SkepForget

Proposition: $Mod(SkepForget(F, x)) = Mod(F) \cap \{Switch(w, x) | w \models F\}$

Let F be a propositional sentence, over a signature $\Sigma = \{a, b\}$, a propositional variable $x \in \Sigma$ and w be an interpretation.

Using structural induction, we can prove that the proposition holds. We consider the following cases:

1. **Base Case:** F is a propositional atomic variable.

   $Mod_{\Sigma_1}(a)$ $\equiv$

a	a
1	1

   $Switch(w, a) | w \models a$ $\equiv$

a	a
0	0

   $Mod_{\Sigma_1}(SkepForget(a))$ $\equiv \emptyset$

   $Mod(F)_{\Sigma_1} \cap \{Switch(w, a) | w \models a\}$ $\equiv \emptyset$

   with $\Sigma_1\{a\}$

2. **Negation Case:**

   $Mod_{\Sigma_1}(\neg a)$ $\equiv$

a	¬ a
0	1

   $Switch(w, a) | w \models \neg a$ $\equiv$

a	¬ a
1	0

   $Mod_{\Sigma_1}(SkepForget(\neg a))$ $\equiv \emptyset$

   $Mod(F)_{\Sigma_1} \cap \{Switch(w, a) | w \models \neg a\}$ $\equiv \emptyset$

   with $\Sigma_1\{a\}$

3. $\vee$ **Case:**

$Mod_{\Sigma_2}((a \vee b))$ $\equiv$

a	b	(a ∨ b)
1	1	1
1	0	1
0	1	1

$Switch(w, a)|w \models (a \vee b)$ $\equiv$

a	b	(a ∨ b)
0	1	1
0	0	0
1	1	1

$Mod_{\Sigma_2}(SkepForget((a \vee b)))$ $\equiv$

a	b	((⊤ ∨ b) ∧ (⊥ ∨ b))
1	1	1
0	1	1

$Mod(F)_{\Sigma_2} \cap \{Switch(w, a)|w \models (a \vee b)\}$ $\equiv$

a	b	(a ∨ b)
1	1	1
0	1	1

with $\Sigma_2\{a, b\}$

4. ∧ **Case:**

$Mod_{\Sigma_2}((a \wedge b))$ $\equiv$

a	b	(a ∧ b)
1	1	1

$Switch(w, a)|w \models (a \wedge b)$ $\equiv$

a	b	(a ∧ b)
0	1	0

$Mod_{\Sigma_2}(SkepForget((a \wedge b)))$ $\equiv \emptyset$

$Mod(F)_{\Sigma_2} \cap \{Switch(w, a)|w \models (a \wedge b)\}$ $\equiv \emptyset$

with $\Sigma_2\{a, b\}$

a	b	¬ a	((⊤ ∧ b) ∧ (⊥ ∧ b))	((⊤ ∨ b) ∧ (⊥ ∨ b))	(¬ ⊤ ∧ ¬ ⊥)	(a ∧ b)	(a ∨ b)
1	1	0	0	1	0	1	1
1	0	0	0	0	0	0	1
0	1	1	0	1	0	0	1
0	0	1	0	0	0	0	0

reference

References

[EKI19] Thomas Eiter and Gabriele Kern-Isberner. A brief survey on forgetting from a knowledge representation and reasoning perspective. *KI-Künstliche Intelligenz*, 33:9–33, 2019.

[LLM03] Jérôme Lang, Paolo Liberatore, and Pierre Marquis. Propositional independence-formula-variable independence and forgetting. *Journal of Artificial Intelligence Research*, 18:391–443, 2003.

[LR94] Fangzhen Lin and Ray Reiter. Forget it. In *Working Notes of AAAI Fall Symposium on Relevance*, pages 154–159, 1994.